

Rapport parallélisme

Nom: Bastien NGUYEN

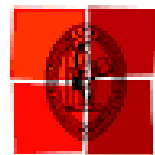
Numéro étudiant: 22300553

Groupe: B11

Exercice 6



UNIVERSITÉ
TOULOUSE III
PAUL SABATIER



Université
de Toulouse

Introduction

Pour rappel j'ai 6 coeur sur ma machine. Tout les résultats que je montre seront fait à partir de ma machine.

Voici la liste de tout les éléments dans mon archive:

- les 3 fichiers de tests: test.gr.txt, rome99.gr.txt et new-york.gr.txt
- un fichier ex6-seq qui est la version séquentielle fournis et que j'ai à peine modifié pour faire un affichage.
- un fichier ex6-par qui est la version parallèle demandée.
- un fichier ex6-moy qui me permet de comparer plusieurs itération de séquentielle et parallèle de façon à obtenir des résultats plus correctes.

Pour mieux comprendre ex6-moy il prend 3 arguments le fichier a traite, le nombre de thread max sur lequel le programme va tester la version parallèle et le nombre d'itération utilisé pour faire la moyenne.

Comme newyork.gr.txt demande beaucoup de calcul je n'ai pas utilisé beaucoup d'itération pour son calcul (seulement 2). Comme la version séquentielle nous est fourni je ne vais pas avoir de parti pour expliquer mon implémentation comme dans les ancien rapport je vais juste montrer les mesures temporelles obtenues.

Séquentielle

Pour faire mes mesures temporelles je décide de partir du début de la fonction djikstra jusqu'à sa fin tout simplement. Comme le sujet nous indique que le fichier test.gr.txt est trop petit pour pouvoir faire des mesures de performances je ne l'ai pas inclus.

Voici donc mes différentes mesures sur les deux fichiers de tests:

Rome	New-York
0,073s	480s

On voit bien que la différences des tailles des fichiers influent grandement sur le temps de calcul. On a même un fichier qui prend plus 8 min à être traité.

On déduit déjà que paralléliser ce programme va nous économiser beaucoup de temps pour new-york.

Parallèle

Pour réaliser la version parallèle de djikstra, j'ai réfléchi à ce qui pouvait être paralléliser ou non.

Les deux parties d'initialisation de d et P sont parallélisables je l'ai donc fait avec un for classique. Cela fait gagné un peu de temps mais c'est les autres parties qui vont le plus influencer sur le temps de calcul.

Pour la partie de recherche de nearest_node et shortest_dist on peut le paralléliser mais il faut bien protéger les deux variables. J'ai d'abord penser à une réduction sur min mais comme il y a deux variables c'est impossible. Ensuite j'ai donc penser à créer des variables local qui en fin de calcul sont comparés en section critique pour éviter d'avoir des problèmes de concurrences. Comme les threads rentrent en section critiques ils n'ont pas besoin d'attendre que les autres arrivent au même endroit qu'eux ce pourquoi je rajoute un nowait pour supprimer la synchronisation implicite du for.

Pour la partie de calcul de d[i] on pourrait penser que on ne peut pas la paralléliser car i peut être égale à nearest_node est donc créer une dépendance mais en fait comme on fait juste avant avant $P[\text{nearest_node}] = 1$ et que le if s'exécute que si $P[i] \neq 1$ alors en fait quand $i = \text{nearest_node}$ alors le if ne s'exécute pas et donc aucune dépendance ne se créer. Pour paralléliser cette partie j'ai donc fait un for classique.

Parallèle

Voici les mesures temporelles que j'ai eu sur cette version:

Nombre de threads	Rome	New-York
1	0,082s	511s
2	0,053s	311s
4	0,044s	221s
6	0,036s	160s
8	0,061s	227s
10	0,182s	303s
12	2,952s	458

On remarque que pour 12 threads à rome on augmente énormément par rapport à 10 threads ce qui peut etre explique par plusieurs choses: mon nombre de coeur et de la cohérence de caches.

Analyse

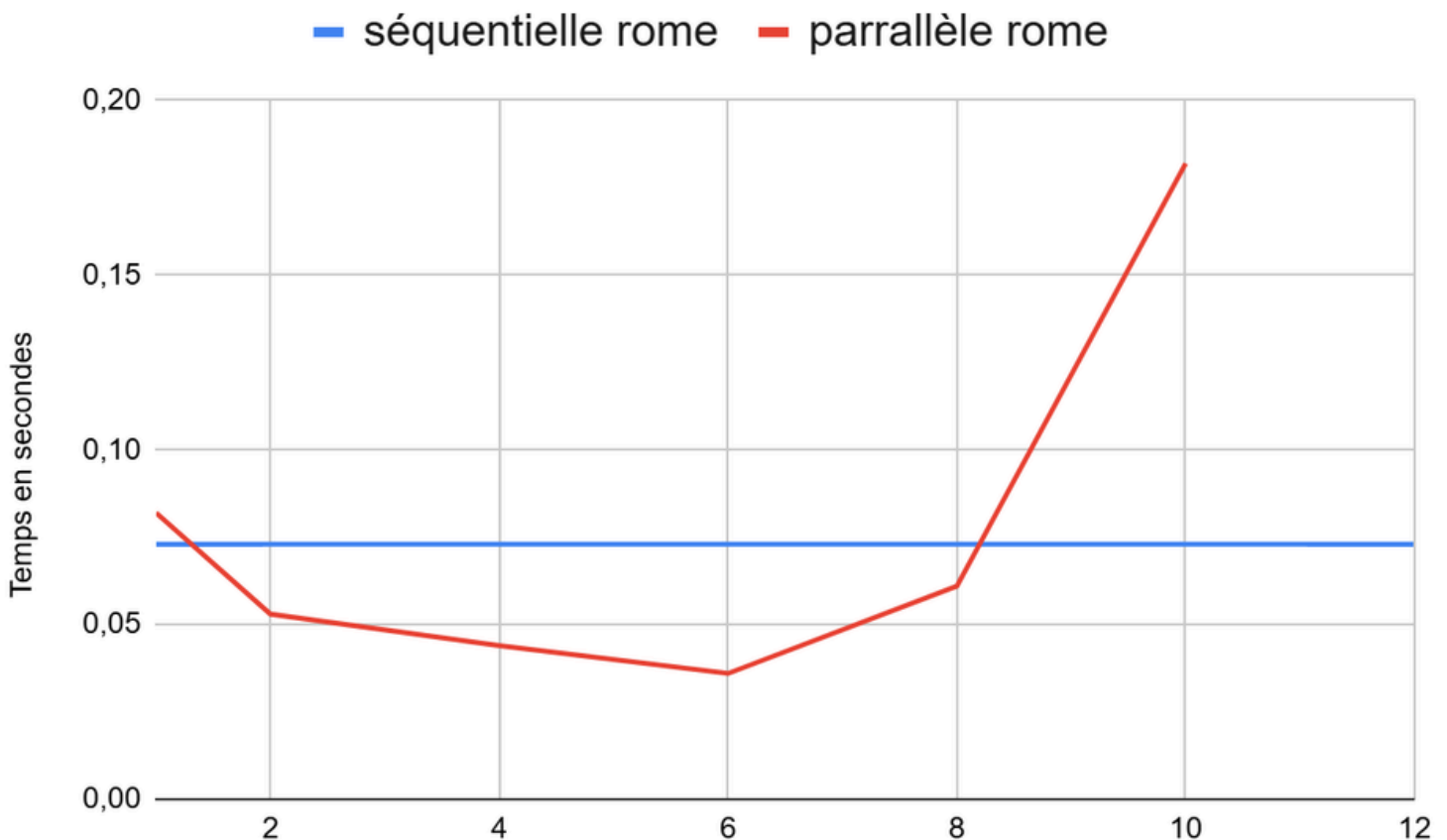
Grace aux mesures des deux versions on peut calculer l'accélération obtenu pour chaque image.

Voici un tableau des différentes accélération obtenue en fonction du nombre de threads.

Nombre de threads	Rome	New-York
1	0,89	0,94
2	1,38	1,54
4	1,66	2,17
6	2,03	3
8	1,20	2,11
10	0,40	1,58
12	0,025	1,05

Analyse

En regardant les mesures d'accélération on peut conclure que plus il y a de calcul à faire plus la version parallèle est rentable mais qu'en dépassant mon nombre de coeur l'accélération ralentit puis même baisse avec trop de threads. On voit même que pour rome il devient plus rentable de rester en séquentielle que d'utiliser 10 ou 12 threads. Cela prouve encore une fois que dépasser le nombre de coeur est une grosse erreur. On peut même modéliser une courbe représentant l'évolution des temps de calculs sur rome (j'ai enlevé 12 threads car sinon le graphique est illisible).

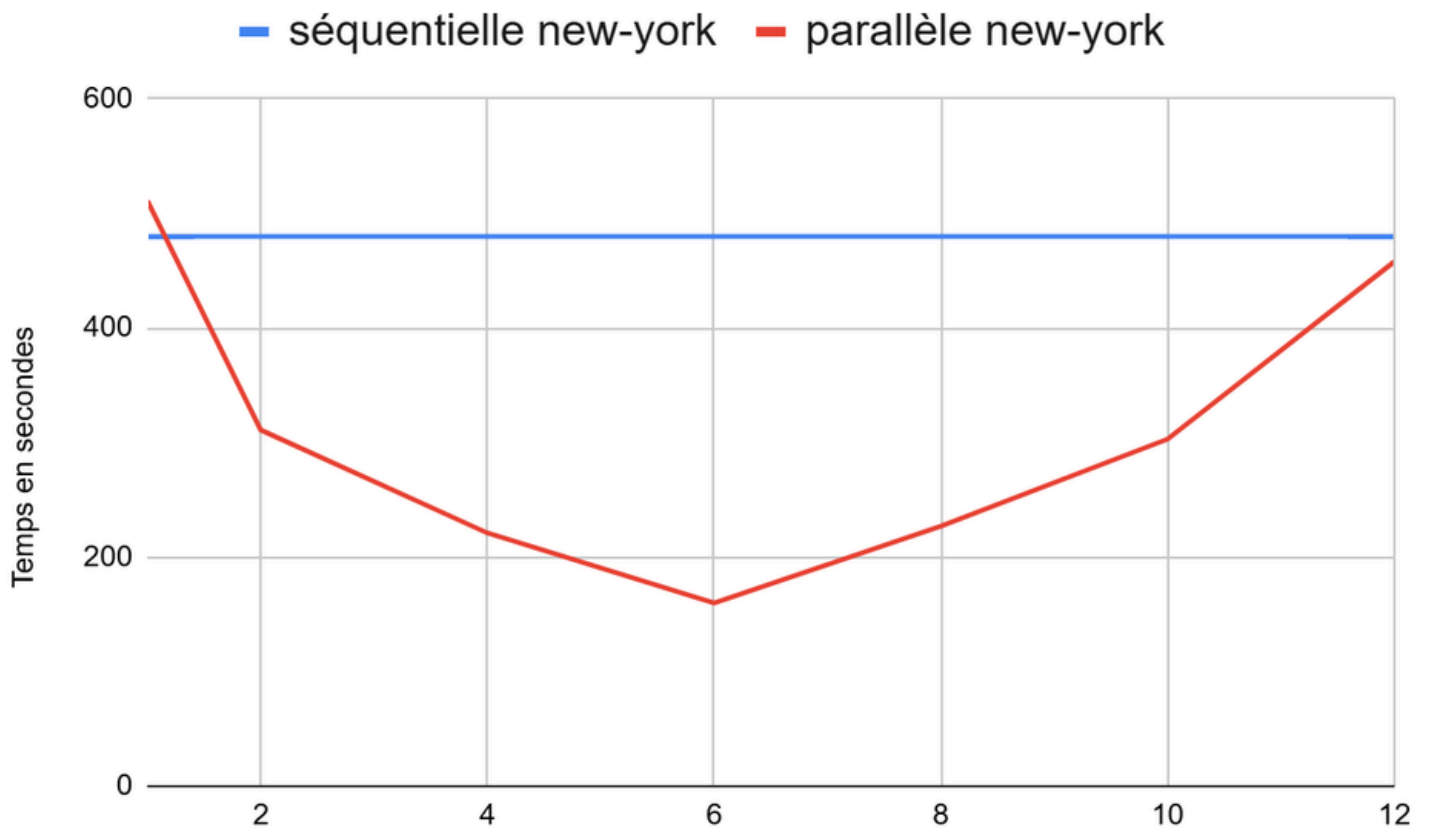


Analyse

Pour New-York, c'est là que l'on comprend que paralléliser un programme est très important car cela permet d'économiser beaucoup de temps.

Je ne sais pas si ma parallélisation est la plus efficace mais pour le coup cela me permet bien d'estimer l'intérêt de la parallélisation.

Voici une courbe représentant le temps d'exécution pour new-york en fonction du nombre de threads. On y voit bien dessus que le cout des primitives openMp influent sur le temps d'exécution.



Conclusion

Ce tp montre enfin un cas où la parallélisation fait gagner du temps dans l'ordre des minutes. On conclue bien qu'avant paralléliser un programme il faut se poser la question est-ce que mon programme va faire beaucoup de calcul. Sur des petites données comme test.gr.txt paralléliser ne va que ralentir le programme alors que sur new-york il est indispensable de paralléliser le programme. Sur Rome la question est donc est ce qu'il est rentable d'utiliser les ressources que demande la parallélisation par rapport au temps gagner.